The Complete Full Stack Developer Course

From Beginner to Professional

தொடக்கநிலையிலிருந்து நிபுணத்துவம் வரை

1. Introduction *(அறிமுகம்)*

Who is a Full Stack Developer?

A Full Stack Developer is a versatile professional who can work on both the **frontend** (client-side) and the **backend** (server-side) of a web application. They have a comprehensive understanding of the entire technology stack, from the user interface to the database.

- **Frontend:** The part of the application that users see and interact with in their browser. *(பயனர்கள் பார்க்கும் மற்றும் தொடர்பு கொள்ளும் பகுதி).*
- **Backend:** The server, application logic, and database that work behind the scenes. *(திரைக்குப் பின்னால் செயல்படும் சர்வர், பயன்பாட்டு தர்க்கம் மற்றும் தரவுத்தளம்).*

Learning Path Overview

This course is designed to be followed sequentially. We will start with the visual part of the web (Frontend), then move to the brains (Backend), the memory (Databases), and finally, the skills needed to manage and deploy your projects.

2. Module 1: Frontend Development *(கிளையன்ட்-சைடு டெவலப்மென்ட்)*

Estimated Time: 8-10 Weeks

HTML5 (HyperText Markup Language)

HTML provides the fundamental structure of web pages. It's the skeleton of your application. *(HTML இணையப் பக்கங்களின் அடிப்படை கட்டமைப்பை வழங்குகிறது. இது உங்கள் பயன்பாட்டின் எலும்புக்கூடு ஆகும்.)*

Key Concepts:

- **Tags & Elements:** `<h1>`, `<p>`, `<a>`, `<img>`, `<div>`, `<span>`
- **Attributes:** `class`, `id`, `src`, `href`
- **Semantic HTML:** Using tags that describe their meaning, like `<header>`, `<footer>`, `<nav>`, `<article>`, `<section>`. This improves SEO and accessibility.
- **Forms:** Collecting user input with `<form>`, `<input>`, `<textarea>`, `<button>`.

Example: Basic HTML Page Structure

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My First Page</title>
</head>
<body>
    <header>
        <nav>
            <a href="/">Home</a>
            <a href="/about">About</a>
        </nav>
    </header>
    <main>
        <h1>Welcome to My Website!</h1>
        <p>This is a paragraph explaining the content.</p>
    </main>
    <footer>
        <p>© 2025 My Website</p>
    </footer>
</body>
</html>
```

CSS3 (Cascading Style Sheets)

CSS is used to style and visually format the HTML content. It controls colors, fonts, layout, and responsiveness. *(CSS ஆனது HTML உள்ளடக்கத்தை ஸ்டைல் செய்யவும் மற்றும் பார்வைக்கு வடிவமைக்கவும் பயன்படுகிறது. இது வண்ணங்கள், எழுத்துருக்கள், தளவமைப்பு மற்றும் பதிலளிக்கும் தன்மையைக் கட்டுப்படுத்துகிறது.)*

Key Concepts:

- **Selectors:** Target HTML elements to apply styles (e.g., `h1`, `.my-class`, `#my-id`).
- **Box Model:** The concept that every HTML element is a rectangular box with margin, border, padding, and content.
- **Flexbox:** A one-dimensional layout model for aligning items in rows or columns. Excellent for component alignment.
- **Grid:** A two-dimensional layout model for creating complex grid-based layouts.
- **Responsive Design:** Using media queries to adapt the layout for different screen sizes (desktops, tablets, phones).
- **Transitions & Animations:** Creating smooth visual effects and animations.

Example: Flexbox Centering & Responsive Design

```
/* Style for a container to center its children */
.container {
    display: flex;
    justify-content: center; /* Horizontally center */
    align-items: center;    /* Vertically center */
    height: 100vh;
    background-color: #f0f0f0;
}

.box {
    width: 200px;
    padding: 20px;
    background-color: dodgerblue;
    color: white;
    text-align: center;
    border-radius: 8px;
}

/* Media Query for mobile devices */
/* (மொபைல் சாதனங்களுக்கான மீடியா கேள்வி) */
@media (max-width: 600px) {
    .container {
        flex-direction: column; /* Stack items vertically on small screens */
    }
    .box {
        width: 90%; /* Take up more width on small screens */
```

```
    }
}
```

JavaScript (ES6+)

JavaScript brings interactivity to your website. It's the programming language of the web. *(ஜாவாஸ்கிரிப்ட் உங்கள் வலைத்தளத்திற்கு ஊடாடும் தன்மையைக் கொண்டுவருகிறது. இது இணையத்தின் நிரலாக்க மொழி.)*

Key Concepts:

- **Variables:** `let` (block-scoped, can be reassigned), `const` (block-scoped, cannot be reassigned).
- **Data Types:** String, Number, Boolean, Object, Array.
- **Functions:** Reusable blocks of code. Arrow functions (=>) provide a shorter syntax.
- **Loops:** `for`, `while`, `forEach`, `map`.
- **DOM Manipulation:** Selecting and modifying HTML elements (e.g., `document.getElementById()`, `element.innerHTML`).
- **Events:** Responding to user actions like `click`, `submit`, `mouseover`.
- **Asynchronous JavaScript:**

  - **Promises:** An object representing the eventual completion (or failure) of an asynchronous operation.

  - **async/await:** Modern syntax to handle promises in a more readable, synchronous-like way.

Example: Fetching Data from an API

```
// Function to get user data from an API
// (API இலிருந்து பயனர் தரவைப் பெறும் செயல்பாடு)
async function fetchUserData() {
    const apiUrl = 'https://jsonplaceholder.typicode.com/users/1';

    try {
        const response = await fetch(apiUrl); // Wait for the network request
        if (!response.ok) {
            throw new Error('Network response was not ok');
        }
```

```
        const userData = await response.json(); // Wait for the JSON parsing

        // Display data on the page
        document.getElementById('user-name').textContent = userData.name;
        document.getElementById('user-email').textContent = userData.email;

    } catch (error) {
        console.error('Fetch error:', error);
        document.getElementById('user-info').textContent = 'Failed to load user
    }
}

// Call the function when the page loads
fetchUserData();
```

React.js

React is a powerful JavaScript library for building dynamic and complex user interfaces using a component-based architecture. *(React என்பது கூறு அடிப்படையிலான கட்டமைப்பைப் பயன்படுத்தி மாறும் மற்றும் சிக்கலான பயனர் இடைமுகங்களை உருவாக்குவதற்கான ஒரு சக்திவாய்ந்த ஜாவாஸ்கிரிப்ட் நூலகமாகும்.)*

Key Concepts:

- **Components:** Reusable, self-contained pieces of UI (e.g., a `Button` or a `UserProfile` component).
- **JSX:** A syntax extension for JavaScript that looks like HTML, used to describe what the UI should look like.
- **Props:** (Properties) Data passed from a parent component to a child component.
- **State:** Data that is managed within a component. When state changes, the component re-renders.
- **Hooks:** Functions that let you "hook into" React state and lifecycle features from function components.
  - `useState`: To manage state.
  - `useEffect`: To perform side effects (like data fetching).
  - `useContext`: To manage global state without passing props down through many levels.

- **Routing:** Using libraries like `react-router-dom` to create multi-page applications.

Example: A Simple Counter Component

```
import React, { useState } from 'react';

// Counter Component (கவுண்டர் கூறு)
function Counter() {
    // 'count' is the state variable, 'setCount' is the function to update it.
    // ('count' என்பது state மாறி, 'setCount' அதை புதுப்பிக்கும் செயல்பாடு
    const [count, setCount] = useState(0);

    return (
        <div>
            <h2>Counter</h2>
            <p>Current count: {count}</p>
            <button onClick={() => setCount(count + 1)}>Increment</button>
            <button onClick={() => setCount(count - 1)}>Decrement</button>
        </div>
    );
}

export default Counter;
```

3. Module 2: Backend Development *(சர்வர்-சைடு டெவலப்மென்ட்)*

Estimated Time: 6-8 Weeks

Node.js

Node.js is a JavaScript runtime environment that allows you to run JavaScript on the server. *(Node.js என்பது ஒரு ஜாவாஸ்கிரிப்ட் இயக்க சூழலாகும், இது சர்வரில் ஜாவாஸ்கிரிப்டை இயக்க அனுமதிக்கிறது.)*

Key Concepts:

- **NPM (Node Package Manager):** A massive ecosystem of open-source packages (libraries) you can use in your projects.
- **Modules:** `require` (CommonJS) and `import`/`export` (ES Modules) to organize and share code between files.

- **File System (`fs`):** A built-in module to interact with the file system (read, write, delete files).
- **Event Loop:** The core of Node.js that handles asynchronous operations efficiently without blocking the main thread.

Example: Reading a File

```
const fs = require('fs');

// Asynchronous file read (ஒத்திசைவற்ற கோப்பு படித்தல்)
fs.readFile('example.txt', 'utf8', (err, data) => {
    if (err) {
        console.error("Error reading the file:", err);
        return;
    }
    console.log("File content:", data);
});

console.log("This message logs first!"); // Demonstrates non-blocking nature
```

Express.js
Express is a minimal and flexible Node.js web application framework that provides a robust set of features for building web and mobile applications, especially APIs.
(எக்ஸ்பிரஸ் என்பது ஒரு குறைந்தபட்ச மற்றும் நெகிழ்வான Node.js வலை பயன்பாட்டு கட்டமைப்பாகும்.)

Key Concepts:

- **Routing:** Defining how an application responds to a client request to a particular endpoint (URI) and a specific HTTP method (GET, POST, etc.).
- **Middleware:** Functions that have access to the request (`req`) and response (`res`) objects, and the `next` function in the application's request-response cycle. Used for logging, authentication, error handling, etc.
- **REST APIs:** Building APIs that follow REST principles for creating, reading, updating, and deleting data (CRUD operations).
- **Authentication:**

  - **JWT (JSON Web Tokens):** A standard for securely transmitting information between parties as a JSON object. Used for stateless authentication.

Example: A Simple Express API Server

```
const express = require('express');
const app = express();
const port = 3000;

// Middleware to parse JSON bodies
app.use(express.json());

// Sample data
let users = [{ id: 1, name: 'Alice' }, { id: 2, name: 'Bob' }];

// GET route to fetch all users
// (அனைத்து பயனர்களையும் பெற GET வழி)
app.get('/api/users', (req, res) => {
    res.json(users);
});

// POST route to add a new user
// (புதிய பயனரைச் சேர்க்க POST வழி)
app.post('/api/users', (req, res) => {
    const newUser = {
        id: users.length + 1,
        name: req.body.name
    };
    users.push(newUser);
    res.status(201).json(newUser);
});

app.listen(port, () => {
    console.log(`Server running at http://localhost:${port}`);
});
```

4. Module 3: Databases *(தரவுத்தளங்கள்)*

Estimated Time: 4-6 Weeks

SQL (MySQL/PostgreSQL)

SQL (Structured Query Language) is used for managing data in a relational database. Data is stored in tables with rows and columns. *(SQL என்பது உறவுநிலை*

தரவுத்தளத்தில் தரவை நிர்வகிக்கப் பயன்படுகிறது. தரவு அட்டவணைகளில் வரிசைகள் மற்றும் நெடுவரிசைகளில் சேமிக்கப்படுகிறது.)

Key Concepts:

- **CRUD Operations:**
  - **Create:** `INSERT INTO`
  - **Read:** `SELECT`
  - **Update:** `UPDATE`
  - **Delete:** `DELETE`
- **Keys:**
  - **Primary Key:** A unique identifier for each record in a table.
  - **Foreign Key:** A key used to link two tables together.
- **Joins:** Combining rows from two or more tables based on a related column (`INNER JOIN`, `LEFT JOIN`).
- **Indexing:** Improving the speed of data retrieval operations.

Example: SQL Queries

```
-- Create a users table
CREATE TABLE users (
    id INT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL
);

-- Insert a new user
INSERT INTO users (username, email) VALUES ('john_doe', 'john.doe@example.com');

-- Select all users
SELECT id, username, email FROM users;

-- Select a user with a specific ID
SELECT * FROM users WHERE id = 1;
```

NoSQL (MongoDB)

NoSQL databases are non-relational and store data in a more flexible format, often as JSON-like documents. MongoDB is a popular document database. *(NoSQL தரவுத்தளங்கள் உறவற்றவை மற்றும் JSON போன்ற ஆவணங்களாக தரவை சேமிக்கின்றன.)*

Key Concepts:

- **Collections:** Equivalent to tables in SQL.
- **Documents:** Equivalent to rows, stored in a BSON (binary JSON) format.
- **Mongoose:** An ODM (Object Data Modeling) library for MongoDB and Node.js. It helps manage relationships between data, provides schema validation, and is used to translate between objects in code and their representation in MongoDB.
- **Aggregation Framework:** For performing multi-stage data processing and analysis.

Example: Mongoose Schema & Model

```
const mongoose = require('mongoose');

// Define a schema (the structure of the document)
// (ஒரு திட்டத்தை வரையறுக்கவும் (ஆவணத்தின் கட்டமைப்பு))
const userSchema = new mongoose.Schema({
    name: {
        type: String,
        required: true
    },
    email: {
        type: String,
        required: true,
        unique: true
    },
    createdAt: {
        type: Date,
        default: Date.now
    }
});

// Create a model from the schema
const User = mongoose.model('User', userSchema);

// --- Using the model to create a new user ---
```

```
async function createUser() {
    try {
        const newUser = new User({
            name: 'Jane Doe',
            email: 'jane.doe@example.com'
        });
        await newUser.save();
        console.log('User created successfully!');
    } catch (error) {
        console.error('Error creating user:', error);
    }
}
```

5. Module 4: Essential Skills *(அத்தியாவசிய திறன்கள்)*

Estimated Time: 3-4 Weeks

Version Control: Git & GitHub

Git is a distributed version control system for tracking changes in source code. GitHub is a platform for hosting and collaborating on Git repositories. *(Git என்பது மூலக் குறியீட்டில் ஏற்படும் மாற்றங்களைக் கண்காணிக்க ஒரு பதிப்புக் கட்டுப்பாட்டு அமைப்பு. GitHub என்பது Git களஞ்சியங்களை ஹோஸ்ட் செய்வதற்கும் ஒத்துழைப்பதற்கும் ஒரு தளமாகும்.)*

Basic Commands:

- `git init`: Initialize a new Git repository.
- `git add <file>`: Stage a file for the next commit.
- `git commit -m "Your message"`: Save your changes to the local repository.
- `git status`: Check the status of your working directory.
- `git push`: Upload local repository content to a remote repository.
- `git pull`: Fetch and download content from a remote repository.
- `git branch <name>`: Create a new branch.
- `git checkout <name>`: Switch to a branch.

Deployment *(பயன்படுத்துதல்)*

Deployment is the process of making your application accessible to users on the internet. *(பயன்படுத்துதல் என்பது உங்கள் பயன்பாட்டை இணையத்தில் பயனர்களுக்கு அணுகும்படி செய்வதாகும்.)*

Key Concepts:

- **Cloud Hosting:** Using services to host your application.
  - **Vercel/Netlify:** Excellent for deploying frontend (React) applications.
  - **Heroku/Render:** Great for deploying full-stack (Node.js + Database) applications.
  - **AWS/Google Cloud:** Powerful, scalable cloud platforms for large applications.
- **Docker:** A platform for developing, shipping, and running applications in containers. Containers package up code and all its dependencies so the application runs quickly and reliably from one computing environment to another.
- **CI/CD (Continuous Integration/Continuous Deployment):** Automating the process of building, testing, and deploying your code. Tools like GitHub Actions can automatically deploy your site whenever you push changes to your main branch.

Testing *(சோதனை)*
Testing ensures your code works as expected and helps prevent bugs.

- **Unit Testing:** Testing individual functions or components in isolation. (e.g., using Jest for React/JavaScript).
- **API Testing:** Using tools like **Postman** or **Insomnia** to manually test your backend API endpoints to ensure they return the correct data and status codes.

6. Module 5: Mini Projects *(சிறு திட்டங்கள்)*

Putting theory into practice is crucial. *(கோட்பாட்டை நடைமுறைப்படுத்துவது மிகவும் முக்கியம்.)*

Project 1: Portfolio Website

- **Tech Stack:** HTML, CSS, JavaScript (React optional)
- **Features:**
  - Clean, responsive design.
  - Sections for "About Me", "Projects", "Skills", and "Contact".

- A working contact form (can use a service like Formspree).

- **Deployment:** Vercel or GitHub Pages.

Project 2: Full Stack Blog

- **Tech Stack:** React, Node.js, Express, MongoDB/PostgreSQL
- **Features:**
  - User authentication (signup/login).
  - Authenticated users can create, edit, and delete their own blog posts.
  - All visitors can view posts.
  - A REST API backend to handle all data operations.
- **Deployment:** Frontend on Vercel, Backend on Render/Heroku.

Project 3: E-Commerce App

- **Tech Stack:** React, Node.js, Express, MongoDB
- **Features:**
  - User authentication and profiles.
  - Product listings with search and filter functionality.
  - Shopping cart functionality.
  - User reviews for products.
  - **(Advanced)** Payment integration with Stripe or Razorpay.
- **Deployment:** Frontend on Vercel, Backend on Render/Heroku.

7. Syllabus & Learning Path *(பாடத்திட்டம் மற்றும் கற்றல் பாதை)*

This is a suggested timeline. Feel free to adjust based on your pace. *(இது ஒரு பரிந்துரைக்கப்பட்ட காலவரிசை. உங்கள் வேகத்தைப் பொறுத்து மாற்றிக்கொள்ளலாம்.)*

Module	Topics	Estimated Time	Exercises
1. HTML5	Tags, Forms, Semantic HTML	1 Week	Build a simple recipe page or a personal tribute page.
2. CSS3	Selectors, Box Model, Flexbox, Grid, Responsive	2-3 Weeks	Style your recipe page. Try to clone the Google homepage layout.
3. JavaScript	Basics, DOM, Events, Async/Await	3-4 Weeks	Build a simple calculator, a to-do list app, or a weather app using an API.
4. React.js	Components, Props, State, Hooks, Routing	3-4 Weeks	Rebuild your to-do list in React. Build the Portfolio project.
5. Node.js & Express.js	Modules, NPM, Routing, Middleware, REST APIs	3-4 Weeks	Build a simple REST API for your to-do list or blog project.
6. Databases	SQL & NoSQL (choose one to start), Mongoose	2-3 Weeks	Connect your blog API to a database to persist data.
7. Git & GitHub	Basic Commands, Workflow	1 Week	Push all your projects to GitHub.
8. Deployment & Testing	Vercel, Heroku, Postman	1-2 Weeks	Deploy your portfolio and blog. Test your blog API with Postman.

Module	Topics	Estimated Time	Exercises
9. Full Stack Projects	Blog, E-commerce App	4-6 Weeks+	Build and deploy the full-stack projects from start to finish.

Good Luck! ***(நல்வாழ்த்துக்கள்!)***